



Università
degli Studi
di Palermo



E05. Installazione ed uso di PyTorch

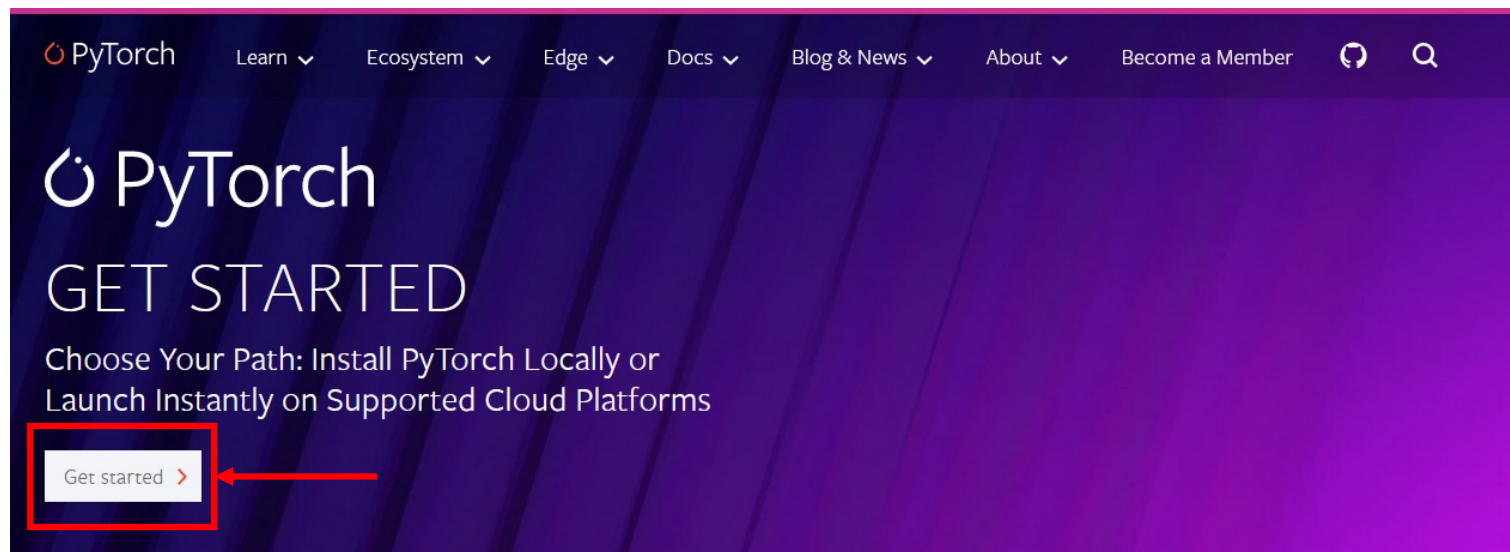
CORSO DI BIG DATA – MODULO MACHINE LEARNING PER I BIG DATA
a.a. 2025/2026

Prof. Roberto Pirrone



Installazione

Pytorch: <https://pytorch.org/>



Università
degli Studi
di Palermo

dipartimento
di ingegneria
unipa



Installazione

NOTE: Latest Stable PyTorch requires Python 3.10 or later.

PyTorch Build	Stable (2.12.0)			Preview (Nightly)	
Your OS	Linux		Mac		Windows
Package	Pip		LibTorch		Source
Language	Python			C++ / Java	
Compute Platform	CUDA 12.6	CUDA 13.0	CUDA 13.2	ROCm 7.2	CPU
Run this Command:	<pre>pip3 install torch torchvision</pre>				



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Pytorch Tensor

Pytorch è costruito sui Pytorch Tensor, degli array n-dimensionali, simili ai numpy array.

NumPy Arrays	PyTorch tensors	Description
<code>numpy.ones(.)</code>	<code>torch.ones(.)</code>	Create an array of ones
<code>numpy.zeros(.)</code>	<code>torch.zeros(.)</code>	Create an array of zeros
<code>numpy.random.rand(.)</code>	<code>torch.rand(.)</code>	Create a random array
<code>numpy.array(.)</code>	<code>torch.tensor(.)</code>	Create an array from given values
<code>x.shape</code>	<code>x.shape</code> or <code>x.size()</code>	Get an array shape



Pytorch Tensor

Un tensore `torch.float32` può essere definito in maniera simile al numpy array

```
x = torch.ones(2, 2)
print(x)
print(x.dtype)
-----
Output
-----
tensor([[1., 1.],
        [1., 1.]])
torch.float32
```

Il data type di può essere cambiato usando il seguente comando

```
x = x.type(torch.uint)
print(x.dtype)
torch.float32
-----
Output
-----
tensor([[1, 1],
        [1, 1]])
torch.uint8
```

Pytorch Tensor

I tensori di torch possono essere convertiti in numpy array (necessari per il funzionamento di altre librerie)

```
y=x.numpy()  
print(y)  
print(y.dtype)  
-----  
Output  
-----  
[[0.80745 0.5727562 ]  
 [0.25486636 0.28319395]]  
dtype('float32')
```

Viceversa se, ad esempio, carichiamo un'immagine con CV2, PIL, Pydicom o altre, avremo un numpy array.

Possiamo convertirla in un torch.tensor utilizzando il seguente comando

```
y = torch.from_numpy(x)  
print(y)  
print(y.dtype)  
-----  
Output  
-----  
tensor([[0.80745 0.5727562 ]  
 [0.25486636 0.28319395]])  
torch.float32
```



Università
degli Studi
di Palermo



Pytorch Tensor

L'utilità di utilizzare i tensori di pytorch, risiede nella possibilità di eseguire calcolo ad alte prestazioni sulla gpu.

Per trasferire un tensore o un batch di tensori nella memoria del dispositivo di calcolo bisognerà definire il *device* da utilizzare per l'esecuzione dello script

```
import torch
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(device)
-----
Output
-----
'cuda' o 'cpu'
```

Pytorch Tensor

Esempio:

```
import nibabel as nib
import torch
from torch import nn
from tqdm import tqdm

volume = nib.load("941_S_6574_3D_MPRAGE.nii.gz")
neural_vol = volume.get_fdata()
print(neural_vol.shape)
print(neural_vol)
-----
Output
-----
(140, 256, 256)
[[[
...
[ 0.  3.  5. ... 10.  8.  6.]
[ 0.  2.  4. ...  8.  6.  6.]
[ 0.  1.  2. ...  3.  4.  8.]]]]
```



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Pytorch Tensor

Esempio:

```
device = 'cpu'
neural_torch = torch.tensor(neural_vol)
neural_torch = neural_torch.to(device)
print(neural_torch)
-----
Output
-----
tensor([[[ 0., 0., 0., ..., 0., 0., 0.],
 [ 0., 1., 1., ..., 2., 2., 1.],
 [ 0., 1., 1., ..., 3., 2., 1.],
 ...,
 [ 0., 3., 5., ..., 10., 8., 6.],
 [ 0., 2., 4., ..., 8., 6., 6.],
 [ 0., 1., 2., ..., 3., 4., 8.]]], dtype=torch.float64)
```

Pytorch Tensor

Esempio:

```
import nibabel as nib
```

```
device = 'cuda'
```

```
neural_torch = torch.tensor(neural_vol)
```

```
neural_torch = neural_torch.to(device)
```

```
print(neural_torch)
```

```
-----
```

Output

```
-----
```

```
tensor([[[ 0., 0., 0., ..., 0., 0., 0.],
```

```
[ 0., 1., 1., ..., 2., 2., 1.],
```

```
[ 0., 1., 1., ..., 3., 2., 1.],
```

```
...,
```

```
[ 0., 3., 5., ..., 10., 8., 6.],
```

```
[ 0., 2., 4., ..., 8., 6., 6.],
```

```
[ 0., 1., 2., ..., 3., 4., 8.] ]], device='cuda:0', dtype=torch.float64)
```



Università
degli Studi
di Palermo



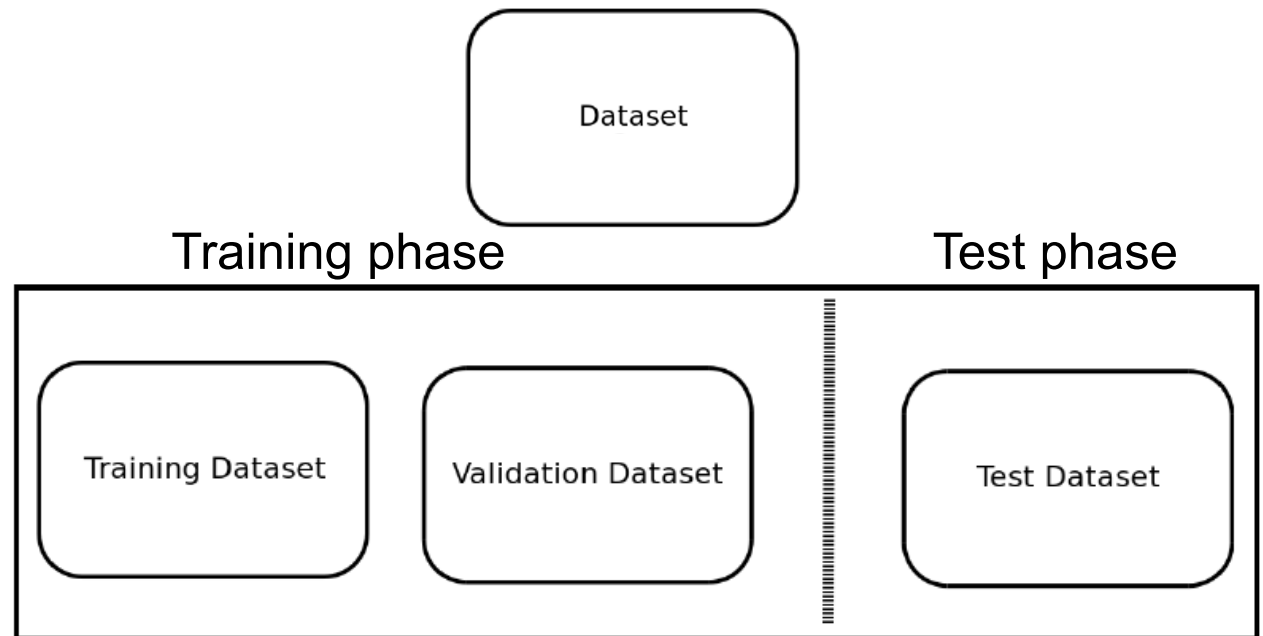
dipartimento
di ingegneria
unipa



Dataloader

Per addestrare e testare il modello, il dataset deve essere suddiviso correttamente favorendo la corretta distribuzione delle etichette (labels)

- `train_test_split`
- K-fold cross-validation
- Stratified cross-validation
- ...



Dataloader

Suddivisi i dati bisogna costruire un *dataloader* per permetterne il corretto caricamento

```
class Dataset(torch.utils.data.Dataset):  
    def __init__(self, paths, targets):  
        self.paths = paths  
        self.targets = targets  
  
    def __len__(self):  
        return len(self.paths)  
  
    def __getitem__(self, index):  
        scan_id = self.paths[index]  
        image = pdc.read_image(scan_id)  
        image = torch.tensor(image).float()  
        y = torch.tensor(self.targets[index], dtype=torch.float)  
        return image, y
```



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Dataloader

Per costruire il dataloader, è necessario definire il *dataset* tramite la classe relativa che può essere estesa per contenere funzioni di manipolazione

```
class Dataset(torch.utils.data.Dataset):
    def __init__(self, paths, targets):
        self.paths = paths
        self.targets = targets

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, index):
        scan_id = self.paths[index]
        image = pdc.read_image(scan_id)
        image = torch.tensor(image).float()
        y = torch.tensor(self.targets[index], dtype=torch.float)
        return image, y
```

proprie del tipo di dato da manipolare.

Ad es. per le immagini:

- Equalizzazione dell'istogramma
- Trasformazioni affini
- Resize
- Padding o Crop

Dataloader

La funzione Dataloader riceverà:

- **Il Dataset**
- **batch_size** → definisce il numero di campioni che verranno propagati attraverso la rete
- **shuffle** → parametro che impone un mescolamento casuale dei dati

```
training_data = Dataset(paths=x, targets=y)

train_loader = torch.utils.data.DataLoader(training_data,
batch_size=batch_size, shuffle=True)

for batch_x, batch_y in tqdm(train_loader):
    batch_x = batch_x.to(device)
    batch_y = batch_y.to(device)
```

Modello

In PyTorch la costruzione del modello utilizza `nn.Module`

```
class Net(nn.Module):  
    def __init__(self):  
        super(Net, self).__init__()  
  
    def forward(self, x):
```



Modello

in `__init__` definisco i layer

In `forward` li
connetto
funzionamente

```
class Net(nn.Module):  
    def __init__(self):  
        super(Net, self).__init__()  
        self.conv1 = nn.Conv2d(1, 20, 5, 1)  
        self.conv2 = nn.Conv2d(20, 50, 5, 1)  
        self.fc1 = nn.Linear(4*4*50, 500)  
        self.fc2 = nn.Linear(500, 10)  
  
    def forward(self, x):  
        x = F.relu(self.conv1(x))  
        x = F.max_pool2d(x, 2, 2)  
        x = F.relu(self.conv2(x))  
        x = F.max_pool2d(x, 2, 2)  
        x = x.view(-1, 4*4*50)  
        x = F.relu(self.fc1(x))  
        x = self.fc2(x)  
        x = funzione_attivazione_uscita(x)  
        return x
```



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Modello

Per visualizzare se il modello sia costruito correttamente

N.B. è necessario installare
torchsummary

```
summary(Net, input_size=(1, 28, 28))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 20, 24, 24]	520
Conv2d-2	[-1, 50, 8, 8]	25,050
Linear-3	[-1, 500]	400,500
Linear-4	[-1, 10]	5,010

Total params: 431,080
Trainable params: 431,080
Non-trainable params: 0

Input size (MB): 0.00
Forward/backward pass size (MB): 0.12
Params size (MB): 1.64
Estimated Total Size (MB): 1.76

Definire la funzione di loss

La scelta della funzione di loss, come sappiamo, determina la corretta stima MLE della distribuzione dei dati insieme all'attivazione del layer di uscita

- **Classificazione**

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = -w_n [y_n \cdot \log x_n + (1 - y_n) \cdot \log(1 - x_n)]$$

- Binary Cross-Entropy

```
torch.nn.BCELoss()
```

- Cross-Entropy

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = - \sum_{c=1}^C w_c \log \frac{\exp(x_{n,c})}{\sum_{i=1}^C \exp(x_{n,i})} y_{n,c}$$

```
torch.nn.CrossEntropyLoss()
```

Softmax inclusa nella loss!!!

Definire la funzione di loss

La scelta della funzione di loss, come sappiamo, determina la corretta stima MLE della distribuzione dei dati insieme all'attivazione del layer di uscita

- **Classificazione**

- Negative Log-Likelihood

```
torch.nn.NLLLoss()
```

*Peso opzionale delle classi
→ Per classi sbilanciate*

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top,$$

$$l_n = -w_{y_n} x_{n, y_n},$$

$$w_c = \text{weight}[c] \cdot 1\{c \neq \text{ignore_index}\},$$

$$\ell(x, y) = \begin{cases} \sum_{n=1}^N \frac{1}{\sum_{n=1}^N w_{y_n}} l_n, & \text{if reduction = 'mean';} \\ \sum_{n=1}^N l_n, & \text{if reduction = 'sum'}. \end{cases}$$

*Softmax non inclusa nella loss!!!
Sono i logit puri
Nell'ultimo layer usare:*

```
torch.nn.LogSoftmax()
```

Definire la funzione di loss

La scelta della funzione di loss, come sappiamo, determina la corretta stima MLE della distribuzione dei dati insieme all'attivazione del layer di uscita

- **Regressione**

- MAE

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = |x_n - y_n|,$$

```
torch.nn.L1Loss()
```

- MSE

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = (x_n - y_n)^2$$

```
torch.nn.MSELoss()
```



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa

$$\ell(x, y) = \begin{cases} \text{mean}(L), & \text{if reduction = 'mean'}; \\ \text{sum}(L), & \text{if reduction = 'sum'}. \end{cases}$$



Training

Completate tutte queste fasi, si può procedere alla costruzione della procedura di addestramento

```
def train(model, train_loader, num_epochs, device, optimizer):  
    for epoch in range(num_epochs):  
        print(f'Epochs n:{epoch+1}/{num_epochs}')  
        # Imposta il modello in fase di apprendimento  
        model.train()  
        for batch_x, batch_y in tqdm(train_loader):  
            batch_x = batch_x.to(device)  
            batch_y = batch_y.to(device)  
            outputs = model(batch_x).to(device) #(n,1)  
            outputs = outputs.squeeze() #(n)  
            # select a loss function  
            criterion = nn.BCELoss()  
            loss = criterion(outputs, batch_y)  
            # Backward and optimize  
            optim = optimizer  
            optim.zero_grad()  
            loss.backward() # esegue la backpropagation  
            optim.step() # ottimizza i pesi ad ogni step
```



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Validation

Alla fine di ogni epoca, per valutare i miglioramenti del modello si utilizza il validation set.

```
model.eval()
with torch.no_grad():
    valid_loss_batch, valid_acc_batch = [], []
    for batchVAL_x, batchVAL_y in tqdm(val_loader):
        batchVAL_x = batchVAL_x.to(device)
        batchVAL_y = batchVAL_y.to(device)
        # validation step
        val_out = model(batchVAL_x) #(n,1)
        val_out = val_out.squeeze() #(n)
        criterion = nn.BCELoss()
        loss_val = criterion(val_out, batchVAL_y)
        valid_loss_batch.append(loss_val.item())
```

Regularizzazione

Early Stopping

```
def early_stopping(loss_values, best_loss, epochs_no_improve):  
    if loss_values < best_loss:  
        print(f"La val_loss è minore della soglia:\t{best_loss}")  
        print(f"La nuova val_loss è la seguente:\t{loss_values}")  
        best_loss = loss_values  
        epochs_no_improve = 0  
        return loss_values, best_loss, epochs_no_improve  
    else:  
        epochs_no_improve += 1  
        return loss_values, best_loss, epochs_no_improve
```



Regularizzazione

Early Stopping

```
model.eval()
with torch.no_grad():
    valid_loss_batch, valid_acc_batch = [], []
    for batchVAL_x, batchVAL_y in tqdm(val_loader):
        batchVAL_x = batchVAL_x.to(device)
        batchVAL_y = batchVAL_y.to(device)
        # validation step
        val_out = model(batchVAL_x)
        val_out = val_out.squeeze()
        criterion = nn.BCELoss()
        loss_val = criterion(val_out, batchVAL_y)
        loss_values, best_loss, epochs_no_improve =
early_stopping(loss_values=loss_val,
                best_loss=start_loss, epochs_no_improve=epochs_no_improve)
        if epochs_no_improve == patience:
            break
```



Regolarizzazione

Model Checkpoint

```
def modelCheckpoint(model, epoch, model_path, optimizer, loss_training, loss_val,
acc_val, fileName):
    """[summary]

    Args:
        model (model): pytorch model to save
        epoch (int): epochs
        model_path (str): path where save model in format "folder/folder/"
        optimizer (--): optimizer used during training
        loss_training (float): value of training loss
        loss_val (float): value of validation loss
        acc_val (float): value of validation accuracy
        fileName (str): name of model

    """
    fileName_weight = f"{fileName}.tar"
    fileName_model = f"{fileName}.pth"
    savePath_name_w = model_path+fileName_weight
    savePath_name_m = model_path+fileName_model
    # salvo i pesi e tutte le info di addestramento
    torch.save({'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'loss_validation': loss_val,
        'accuracy_validation': acc_val,
        'loss_training': loss_training
    }, savePath_name_w)
    # salvo l'intero modello
    torch.save(model, savePath_name_m)
```



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa

